Tutorials

Welcome to the UnifiedLoot tutorials! This section provides hands-on guides for common game development scenarios.

- **The Goblin Encounter**: A complete, end-to-end example featuring custom definitions, factories, and complex pipelines.
- **Boss Loot Systems**: How to handle multi-stage drops and guaranteed item types.
- **Gacha & Loot Boxes**: Implementing pity systems and drop rate modifiers.
- **Dynamic Scaling**: Adjusting loot quality based on player stats or game difficulty.

Example Projects

You can find the complete source code for the tutorials in the package's `Assets/UnifiedLoot/Samples~` directory.

Tutorial: The Goblin Encounter

This tutorial walks through the `GoblinEncounterExample` sample, demonstrating a real-world RPG loot system. We'll cover custom item definitions, complex pipelines, and advanced features like pity systems and luck-based scaling.

1. Defining the Loot

In a real project, loot is rarely just an `enum`. You often need metadata like item names, colors, icons, and base values.

The Item Definition

We start by defining a `GoblinItemDef` which holds the static data for our items. This is what you would typically configure as a `ScriptableObject` or a serializable class in a list.

```
public enum Rarity {
    Common,
    Rare,
    Legendary
}

[System.Serializable]
public class GoblinItemDef {
    public string ItemName = "New Item";
    public Color Color = Color.white;
    public int BaseValue = 10;
    public string IconUnicode = "📦";
    public Rarity Rarity = Rarity.Common;

    public override string ToString() => $"{IconUnicode} {ItemName}";
}
```

The Runtime Instance

When loot drops, we might want to create a unique instance of that item, potentially with randomized stats (e.g., varying gold value).

```
public class GoblinItemInstance {
    public string Name { get; set; } = string.Empty;
    public Color Color { get; set; }
    public int Value { get; set; }
    public string Icon { get; set; } = string.Empty;
    public Rarity Rarity { get; set; }
```

```
    public override string ToString() => $"{Icon} {Name} ({Value}g)";
}
```

---

2. Setting Up the Tables

UnifiedLoot supports designer-friendly `ScriptableObject` assets. To allow designers to edit tables in the Inspector, we create a simple class inheriting from `LootTableAsset<T>`.

```
using NS.UnifiedLoot;
using UnityEngine;

namespace NS.UnifiedLoot.Examples {
    /// <summary>
    /// Designer-editable loot table for the goblin encounter example.
    ///
    /// <para><b>How to create an asset:</b><br/>
    /// Right-click in the Project window → <c>Create → UnifiedLoot/Examples/Goblin
Item Table</c>.
    /// </para>
    ///
    /// <para><b>How to fill it in:</b><br/>
    /// Expand the Entries list and add rows.  Each row has:
    /// <list type="bullet">
    ///   <item><b>Item</b> — assign a <see cref="GoblinItemDef"/> asset.</item>
    ///   <item><b>Weight</b> — relative drop chance (higher = more common).
The Inspector
    ///         shows the effective % next to each weight field.</item>
    ///   <item><b>Quantity</b> — min/max quantity rolled when this entry
drops.</item>
    /// </list>
    /// </para>
    ///
    /// <para><b>Assign the asset</b> to the matching slot on <see
cref="GoblinEncounterExample"/>
    /// and it will override the code-defined fallback table at runtime.</para>
    /// </summary>
    [CreateAssetMenu(menuName = "UnifiedLoot/Examples/Goblin Item Table", fileName =
"New Goblin Item Table")]
    public class GoblinItemTable : LootTableAsset<GoblinItemDef> { }
}
```

---

3. The Goblin Pipeline

For a regular Goblin, we want a simple but dynamic loot drop:

1. **Luck Bonus**: Rare items should have a higher chance to drop if the player is lucky.
2. **Multiple Rolls**: A goblin drops 3 items.
3. **Bonus Roll**: Lucky players have a chance to get a 4th item.
4. **Consolidation**: If multiple stacks of coins drop, they should be merged into one result.

```
private void BuildGoblinPipeline() {
    //  1. ModifyWeight — rare items (SteelSword, MagicStaff) get a luck bonus
    //  2. WeightedRandom(3) — three rolls per kill
    //  3. BonusRoll — luck-based chance at a 4th free roll
    //  4. Consolidate — stack repeated Coins / Potions
    _goblinPipeline = new LootPipeline<GoblinItemDef>()
        .WithRandom(UnityRandom.Instance)
        .AddStrategy(new ModifyWeightStrategy<GoblinItemDef>((entry, ctx) => {
            var item = entry.Entry.Item;
            if (item is not { Rarity: >= Rarity.Rare })
                return entry.Weight;

            var luck = Mathf.Clamp01(ctx.GetOrDefault(Luck));
            return entry.Weight * (1f + luck * 3f); // up to 4× at max luck
        }))
        .AddStrategy(new WeightedRandomStrategy<GoblinItemDef>(rollCount: 3))
        .AddStrategy(new BonusRollStrategy<GoblinItemDef>(Luck))
        .AddStrategy(new ConsolidateResultsStrategy<GoblinItemDef>())
        .AddObserver(new ConsoleObserver<GoblinItemDef>("Goblin"));
}
```

---

4. The Goblin Captain (Advanced)

The Captain uses a more complex pipeline with **Advanced Scaling**.

Luck-based Scaling

The Captain's pipeline uses luck to significantly increase the weight of legendary items.

```
private void BuildCaptainPipeline() {
    //  1. ModifyWeight — GoblinKingsCrown weight scales with player luck
    //  2. WeightedRandom(1) — single roll from the Flattened table

    _captainPipeline = new LootPipeline<GoblinItemDef>()
        .WithRandom(UnityRandom.Instance)
```

```
        .AddStrategy(new ModifyWeightStrategy<GoblinItemDef>((entry, ctx) => {
            var item = entry.Entry.Item;
            if (item is not { Rarity: >= Rarity.Legendary })
                return entry.Weight;

            var luck = Mathf.Clamp01(ctx.GetOrDefault(Luck));
            return entry.Weight * (1f + luck * 3f);
        }))
        .AddStrategy(new WeightedRandomStrategy<GoblinItemDef>(rollCount: 1))
        .AddObserver(new ConsoleObserver<GoblinItemDef>("Captain"));
}
```

---

5. Building the Loot (The Factory)

UnifiedLoot separates *selection* (what drops) from *instantiation* (creating the object). We use an `ILootFactory` to turn our `GoblinItemDef` into a `GoblinItemInstance`.

```
/// <summary>
/// Factory method to create runtime instances from definitions.
/// Demonstrates how to inject logic (like luck-based value scaling) during the
build process.
/// </summary>
public GoblinItemInstance Create(GoblinItemDef definition, Context context, IRandom
random) {
    return new GoblinItemInstance {
        Name = definition.ItemName,
        Color = definition.Color,
        Icon = definition.IconUnicode,
        Rarity = definition.Rarity,
        Value = Mathf.RoundToInt(definition.BaseValue * random.Range(0.9f, 1.1f) *
(1f + context.GetOrDefault(Luck) * 0.5f))
    };
}
```

---

6. Execution

Finally, we trigger the roll. We use `ExecuteAndBuild` to perform the selection and the instantiation in one go.

```
[ContextMenu("Kill Goblin (roll loot)")]
public List<BuiltLootResult<GoblinItemDef, GoblinItemInstance>> KillGoblin() {
    UpdateContext();
```

```
    _context.Set(Source, "Goblin");
    _reusableResults.Clear();
    _goblinPipeline.ExecuteAndBuild(_activeGoblinTable, this,
_reusableResults, _context);
    LogBuiltResults("Goblin", _reusableResults);
    return _reusableResults;
}
```

Summary of Features Shown

- **LootTableAsset**: Creating designer-editable tables.
- **ModifyWeightStrategy**: Dynamically changing weights based on Context.
- **PityStrategy**: Implementing "bad luck protection".
- **ILootFactory<TDef, TInstance>**: Decoupling data from runtime objects.
- **ExecuteAndBuild**: End-to-end loot generation.

Tutorial: Boss Loot Systems

This tutorial explores the `BossLootExample` sample, showing how to handle high-value encounters where loot is often dropped in multiple stages and certain item types must be guaranteed.

1. Multi-Stage Loot

In complex encounters like bosses, you often don't want a single roll. Instead, you might want to guarantee different categories of loot:

1. **Currency**: A fixed amount or a guaranteed roll for gold/gems.
2. **Materials**: Several rolls from a crafting table.
3. **Equipment**: A high-stakes roll for a piece of gear.

By using multiple pipelines or executing the same pipeline against different tables, you can easily create these sequences.

```
[ContextMenu("Kill Boss (Multi-Stage Loot)")]
public void KillBoss() {
    _allDrops.Clear();
    Debug.Log("<color=red><b>BOSS KILLED!</b></color> Starting loot sequence...");

    // Stage 1: Currency
    if (currencyTable != null)
        _currencyPipeline.ExecuteAndBuild(currencyTable.ToTable(), this,
_allDrops, _context);

    // Stage 2: Materials
    if (materialTable != null)
        _materialPipeline.ExecuteAndBuild(materialTable.ToTable(), this,
_allDrops, _context);

    // Stage 3: Equipment
    if (equipmentTable != null)
        _equipmentPipeline.ExecuteAndBuild(equipmentTable.ToTable(), this,
_allDrops, _context);

    LogResults();
}
```

2. Guaranteed Item Types

One common requirement is ensuring the player gets *at least one* item from a table, even if the table contains "Empty" results or low probabilities. The `GuaranteedDropStrategy` is perfect for this.

```
_equipmentPipeline = new LootPipeline<BossItemDef>()
    .WithRandom(UnityRandom.Instance)
    .AddStrategy(new WeightedRandomStrategy<BossItemDef>(rollCount: 1))
    // If the equipment table had "Empty" entries or low drop chances,
    // this ensures we still get one item from the table.
    .AddStrategy(new GuaranteedDropStrategy<BossItemDef>())
    .AddObserver(new BossLootObserver("Equipment"));
```

If the `WeightedRandomStrategy` results in 0 items (e.g., if all weights failed or an "Empty" entry was rolled), `GuaranteedDropStrategy` will perform an additional roll that ignores "Empty" entries to ensure a result.

3. Implementation Details

Item Definitions

For this example, we use a `BossItemDef` that includes a rarity-based power calculation during the build process.

```
public enum BossItemRarity {
    Common,
    Rare,
    Legendary,
    Epic,
    Artifact
}

[System.Serializable]
public class BossItemDef {
    public string ItemName = "Boss Drop";
    public Color Color = Color.white;
    public BossItemRarity Rarity = BossItemRarity.Common;
    public string IconUnicode = "??";
    public bool IsWeapon;

    public override string ToString() => $"{IconUnicode} {ItemName}";
}
```

The Factory

The factory calculates the `Power` of the item instance based on its rarity and a small random variance.

```
public BossItemInstance Create(BossItemDef definition, Context context, IRandom
random) {
    var powerBase = definition.Rarity switch {
        BossItemRarity.Common => 10,
        BossItemRarity.Rare => 25,
        BossItemRarity.Epic => 50,
        BossItemRarity.Legendary => 100,
        BossItemRarity.Artifact => 250,
        _ => 0
    };

    return new BossItemInstance {
        Name = definition.ItemName,
        Color = definition.Color,
        Rarity = definition.Rarity,
        Icon = definition.IconUnicode,
        Power = powerBase + random.Range(0, 10)
    };
}
```

Summary

Boss loot systems demonstrate:

- **Sequential Execution**: Running multiple loot rolls into a single result set.
- **Fallbacks**: Using `GuaranteedDropStrategy` for player satisfaction.
- **Logic Injection**: Using the factory to scale item stats at runtime.

Tutorial: Gacha & Loot Boxes

This tutorial explores the `GachaExample` sample, showing how to implement common gacha mechanics like hard/soft pity and multi-pulls using UnifiedLoot. We'll also see the correct way to reuse a `Context` across multiple rolls.

1. Context Reuse

When performing multiple rolls (like a "10-pull"), it is more efficient to reuse a single `Context` instance. This avoids unnecessary allocations and allows strategies and observers to persist data more easily throughout the sequence.

```
[ContextMenu("Single Pull")]
public void SinglePull() {
    _recentPulls.Clear();
    _gachaPipeline.ExecuteAndBuild(bannerTable!.ToTable(), this,
_recentPulls, _context);

    LogResults("Single Pull");
}

[ContextMenu("Multi Pull (10x)")]
public void MultiPull() {
    _recentPulls.Clear();

    Debug.Log("<color=orange><b>Multi-Pull Start!</b></color>");
    for (var i = 0; i < 10; i++)
        _gachaPipeline.ExecuteAndBuild(bannerTable!.ToTable(), this,
_recentPulls, _context);

    LogResults("10-Pull");
}
```

2. Cohesive Pity System

Instead of splitting pity logic across multiple unrelated classes, we can implement a cohesive `GachaPitySystem` that handles all stages of the gacha lifecycle. By implementing both `ILootStrategy` and `ILootObserver`, it can manage state, modify weights, and enforce guarantees in one place.

```
/// <summary>
/// A cohesive pity system that handles soft pity weight modification,
/// hard pity enforcement, and success/failure tracking.
```

```
///
/// <para>By implementing both <see cref="ILootStrategy{T}"/> and
<see cref="ILootObserver{T}"/>,
/// it can be added to the pipeline once to handle multiple stages of the
process.</para>
/// </summary>
private class GachaPitySystem : ILootGeneratorStrategy<GachaItemDef> {
    private const int Key4 = 4;
    private const int Key5 = 5;

    private readonly IPityTracker _tracker = new PityTracker();
    private readonly int _threshold4;
    private readonly int _threshold5;
    private readonly int _softPityStart5;


    public GachaPitySystem(int threshold4, int threshold5, int softPityStart5) {
        _threshold4 = threshold4;
        _threshold5 = threshold5;
        _softPityStart5 = softPityStart5;
    }

    public int Pity5 => _tracker.GetFailures(Key5);
    public int Pity4 => _tracker.GetFailures(Key4);

    /// <summary>
    /// Logic for soft pity weight modification.
    /// </summary>
    public float GetSoftPityWeight(WeightedEntry<GachaItemDef> we, Context
context) {
        if (we.Entry.Item?.Rarity != GachaRarity.FiveStar)
            return we.Weight;

        var pity = _tracker.GetFailures(Key5);
        if (pity < _softPityStart5)
            return we.Weight;

        // Increase weight linearly after soft pity start
        var boost = 1f + (pity - _softPityStart5) * 10f;
        return we.Weight * boost;
    }

    /// <summary>
    /// Logic for hard pity enforcement (guaranteed drops).
    /// Runs after the main roll strategy.
    /// </summary>
```

```
    public void Process(LootWorkingSet<GachaItemDef> workingSet, Context context) {
        // If the roll didn't happen or produced nothing, we can't enforce pity here
        if (workingSet.Results.Count == 0)
            return;

        var pity5 = _tracker.GetFailures(Key5);
        var pity4 = _tracker.GetFailures(Key4);

        // 1. Check 5-star hard pity (must happen first as it's higher priority)
        var got5Star = false;
        if (
            pity5 >= _threshold5 - 1 &&
            workingSet.Results.All(r => r.Item.Rarity != GachaRarity.FiveStar)
        ) {
            workingSet.Results.Clear();
            got5Star = TryRollFiltered(workingSet, GachaRarity.FiveStar);
        }

        // 2. Check 4-star hard pity (only if no 5-star or 4-star dropped)
        var hasHighRarity = !got5Star && workingSet.Results.Any(r => r.Item.Rarity
>= GachaRarity.FourStar);
        var got4Star = false;
        if (!hasHighRarity && pity4 >= _threshold4 - 1) {
            workingSet.Results.Clear();
            got4Star = TryRollFiltered(workingSet, GachaRarity.FourStar);
        }

        // 5-star resets on 5-star drop, otherwise increments
        _tracker.Record(Key5, got5Star ? PityResult.Success : PityResult.Failure);

        // 4-star resets on either 4-star or 5-star drop, otherwise increments
        _tracker.Record(Key4, got5Star || got4Star ? PityResult.Success
: PityResult.Failure);
    }

    private bool TryRollFiltered(LootWorkingSet<GachaItemDef> workingSet,
GachaRarity rarity) {
        var filteredEntries = workingSet.WeightedEntries
            .Where(we => we.Entry.Item != null && we.Entry.Item.Rarity == rarity)
            .ToList();

        if (filteredEntries.Count == 0)
            return false;

        var totalWeight = filteredEntries.Sum(we => we.Weight);
        var roll = workingSet.Random.Range(0f, totalWeight);
```

```
        var current = 0f;

        foreach (var we in filteredEntries) {
            current += we.Weight;
            if (!(roll <= current))
                continue;
            workingSet.AddResult(we.Entry, we.Index, totalWeight > 0 ? roll /
totalWeight : 0);
            return true;
        }

        // Fallback
        var first = filteredEntries[0];
        workingSet.AddResult(first.Entry, first.Index, 1f);
        return true;
    }
}
```

Soft Pity

Soft pity increases the drop rate of rare items after a certain threshold of failures. We use `ModifyWeightStrategy` and pass it a method from our pity system that calculates the boost based on the internal `IPityTracker`.

```
private void Awake() {
    // 1. Initialize the pity system (holds state internally via IPityTracker)
    _pitySystem = new GachaPitySystem(hardPity4Star, hardPity5Star,
softPityStart5Star);

    // 2. Reuse a single context for efficiency and data persistence across pulls
    _context = new Context();

    // 3. Build the gacha pipeline
    _gachaPipeline = new LootPipeline<GachaItemDef>()
        .WithRandom(UnityRandom.Instance)
        // Soft Pity: Modify weight of 5-star items using the pity system's
current state
        .AddStrategy(new ModifyWeightStrategy<GachaItemDef>
(_pitySystem.GetSoftPityWeight))
        // Main roll
        .AddStrategy(new WeightedRandomStrategy<GachaItemDef>(rollCount: 1))
        // Hard Pity: Force results if threshold reached and update pity state
        .AddStrategy(_pitySystem);
}
```

Hard Pity

Hard pity guarantees a specific result after a fixed number of failures. The `GachaPitySystem` checks the results of the roll and, if the threshold is reached without a high-rarity drop, it clears the current results and forces a filtered roll from the table.

Summary

The Gacha example demonstrates:

- **Context Reuse**: Reusing a single `Context` instance across multiple rolls for efficiency and data persistence.
- **Dynamic Weighting**: Using `ModifyWeightStrategy` for soft pity ramps.
- **Cohesive Pity System**: Implementing multiple interfaces in a single class to manage state, modify weights, and enforce hard pity.
- **State Management**: Using `IPityTracker` and `ILootObserver` to synchronize persistent state with loot results.

Tutorial: Dynamic Scaling

This tutorial explores the `ScalingExample` sample, showing how to adjust loot quality, quantity, and item stats dynamically based on the game's context (e.g., player level or difficulty).

1. Dynamic Weighting (Quality Scaling)

As a player levels up, you often want them to find better gear. Instead of creating a new loot table for every level, you can use `ModifyWeightStrategy` to adjust the probabilities of an existing table.

```
private void Awake() {
    // Build the scaling pipeline
    _scalingPipeline = new LootPipeline<ScalingItemDef>()
        .WithRandom(UnityRandom.Instance)
        // 1. Dynamic Weighting: Rare items get more likely as level increases
        .AddStrategy(new ModifyWeightStrategy<ScalingItemDef>((entry, ctx) => {
            var level = ctx.GetOrDefault(PlayerLevel, 1);
            var rarity = entry.Entry.Item?.rarity ?? ScalingRarity.Common;

            return rarity switch {
                ScalingRarity.Poor => entry.Weight / (1f + level * 0.1f),
                ScalingRarity.Rare => entry.Weight * (1f + level * 0.05f),
                ScalingRarity.Epic => entry.Weight * (1f + level * 0.025f),
                _ => entry.Weight
            };
        }))
        // 2. Main roll
        .AddStrategy(new WeightedRandomStrategy<ScalingItemDef>(rollCount: 2))
        // 3. Dynamic Quantity: More loot on higher difficulties
        .AddStrategy(new ModifyQuantityStrategy<ScalingItemDef>((qty, ctx) => {
            var diff = ctx.GetOrDefault(Difficulty, 1f);
            return Mathf.RoundToInt(qty * diff);
        }))
        .AddStrategy(new ConsolidateResultsStrategy<ScalingItemDef>());
}
```

In this example, the weights of "Poor" items decrease while "Rare" and "Epic" items increase as the `PlayerLevel` in the `Context` goes up.

2. Dynamic Quantity (Difficulty Scaling)

For harder encounters or higher game difficulties, you might want to drop *more* items. The `ModifyQuantityStrategy` allows you to scale the number of items produced by the generator.

```
private void Awake() {
    // Build the scaling pipeline
    _scalingPipeline = new LootPipeline<ScalingItemDef>()
        .WithRandom(UnityRandom.Instance)
        // 1. Dynamic Weighting: Rare items get more likely as level increases
        .AddStrategy(new ModifyWeightStrategy<ScalingItemDef>((entry, ctx) => {
            var level = ctx.GetOrDefault(PlayerLevel, 1);
            var rarity = entry.Entry.Item?.rarity ?? ScalingRarity.Common;

            return rarity switch {
                ScalingRarity.Poor => entry.Weight / (1f + level * 0.1f),
                ScalingRarity.Rare => entry.Weight * (1f + level * 0.05f),
                ScalingRarity.Epic => entry.Weight * (1f + level * 0.025f),
                _ => entry.Weight
            };
        }))
        // 2. Main roll
        .AddStrategy(new WeightedRandomStrategy<ScalingItemDef>(rollCount: 2))
        // 3. Dynamic Quantity: More loot on higher difficulties
        .AddStrategy(new ModifyQuantityStrategy<ScalingItemDef>((qty, ctx) => {
            var diff = ctx.GetOrDefault(Difficulty, 1f);
            return Mathf.RoundToInt(qty * diff);
        }))
        .AddStrategy(new ConsolidateResultsStrategy<ScalingItemDef>());
}
```

Here, the final quantity is multiplied by a `Difficulty` value provided in the context.

3. Stat Scaling (The Factory)

Finally, the items themselves should be more powerful at higher levels. Since UnifiedLoot separates selection from instantiation, the `ILootFactory` is the perfect place to inject this logic.

```
public ScalingItemInstance Create(ScalingItemDef definition, Context context,
IRandom random) {
    var level = context.GetOrDefault(PlayerLevel, 1);
    var diff = context.GetOrDefault(Difficulty, 1f);

    // Scale power based on base range, player level, and difficulty
    float basePower = random.Range(definition.minBasePower,
```

```
definition.maxBasePower);
    var finalPower = Mathf.RoundToInt(basePower * (1f + level * 0.1f) * diff);

    return new ScalingItemInstance {
        Name = definition.itemName,
        Rarity = definition.rarity,
        Color = definition.color,
        Power = finalPower
    };
}
```

The factory receives the `Context`, allowing it to read the `PlayerLevel` and `Difficulty` to calculate the final `Power` of the `ScalingItemInstance`.

4. Execution

To trigger the roll, we pack the current game state into a `Context` and call `ExecuteAndBuild`.

```
[ContextMenu("Roll Scaling Loot")]
public void RollLoot() {
    _context
        .Set(PlayerLevel, playerLevel)
        .Set(Difficulty, gameDifficulty);

    _results.Clear();
    _scalingPipeline.ExecuteAndBuild(equipmentTable!.ToTable(), this,
_results, _context);

    LogResults();
}
```

Summary

Dynamic scaling demonstrates:

- **Contextual Awareness**: Using the `Context` to pass game state into the loot system.
- **Probabilistic Adjustment**: Changing weights at runtime without modifying assets.
- **Factory-based Scaling**: Decoupling item data from its runtime power level.